CORNERSTONE PDK Monitor — User Manual

Version 1.1.0

A complete guide to the CORNERSTONE PDK Monitor: a single, self-contained HTML application for managing and sharing open-source integrated-photonics Process Design Kits (PDKs). This manual covers everything from first launch to publishing, cloud storage, running scripts, and administering the dashboard.

Table of contents

1. What it is
2. Quick start
3. Modes and access
4. The header and navigation
5. Building blocks and their tabs
6. Getting the data — Distribution
7. Importing a platform from GitHub
8. Cloud storage — fetching large data
9. Cloud storage — uploading (admins)
10. Working with files in a BB
11. Layout / GDS viewer
12. Plots and figures
13. Scripts — editor and runner
14. Comments and notes
15. Uploads view
16. Finding and removing duplicates
17. Multi-select and bulk delete
18. Analysis and AI Insight
19. Managing lists and statuses
20. Publishing and the repo-size model
21. Backups, export, and cleanup
22. Tips and troubleshooting

1. What it is

The CORNERSTONE PDK Monitor is the gateway between photonic designers and fabs: one place to discover, use, and contribute open PDK building blocks. The entire application is a single `.html` file — there is no server and no build step. Open it in a modern browser, or host it on GitHub Pages, SharePoint, or an internal server.

The dashboard organises **building blocks (BBs)** — waveguides, MMIs, rings, modulators, grating couplers, and so on — into **platforms** (e.g. SOI 220 nm) and **tabs**. Each BB carries:

- **Layout** — GDS / OASIS / SVG, plus a sketch image
- **S-matrix** — S-parameter datasets
- **Tests** — measurement / characterisation data
- **Fabrication** — process notes and data
- **Scripts** — Python and tools
- **Documents** — datasheets, manuals, PDFs

Data lives locally in your browser (IndexedDB) and is shared with the team through GitHub and/or any cloud provider.

---

2. Quick start

1. **Open** `cornerstone-pdk-monitor-v1.1.0.html` in Chrome, Edge, Firefox, or Safari. It starts in **read-only** mode — you can browse everything and change nothing.
2. **Load the live data.** Click ⬇ **Distribution → Fetch all enabled sources** to pull the published `dashboard.json` (and contributors' slices) from GitHub. No account or token is needed for public repos.
3. **Explore.** Click any BB to open its Details, Layout, S-matrix, Tests, Fabrication, Scripts, and Documents tabs.
4. **Contribute.** Sign in via ⚙ **Configuration → ⬇ GitHub account** (paste a Personal Access Token) to unlock user-admin or admin features, then add and publish your own BBs and files.

---

3. Modes and access

The dashboard has three modes, shown on the mode pill at the right of the header:

- **Read-only** (default) — browse everything, fetch from GitHub and cloud, leave comments. No sign-in.
- **User-admin** — contribute your own BBs and files and publish a personal *slice*. **Sign in with your own GitHub Personal Access Token (PAT)** — no shared password. The token is verified against GitHub, your GitHub login becomes your identity, and GitHub's own permissions decide what you can actually publish. On first sign-in the dashboard offers to create your `users/<login>/` folder in the repo. (Shared "guest passwords" have been retired; nothing secret travels in `dashboard.json`.)
- **Admin** — full control: curate the contributor list, manage platforms and lists, and publish the canonical dashboard. Reserved for the single curator who owns the canonical dashboard, who signs in with the **admin password** (entered in a small section at the bottom of the Sign-in screen). Contributors do not need this.

The Sign-in screen leads with **GitHub PAT sign-in** as the principal path; the **owner admin password** sits in a separate section at the bottom. There are no shared user-admin passwords — to grant someone contributor access, add them to the GitHub repo (or let them fork and open a PR).

One folder per GitHub account — created automatically. When you sign in with a PAT, the dashboard checks the repo's live `users/` folders (not the local contributor list) to see whether you already have a `users/<login>/` folder. If you don't, it offers to create it for you:

- If your token **can push** to the source repo, it commits the folder there directly.
- If it **can't push** (the usual case for external contributors), it offers to **fork** the repo to your account, create the folder in your fork, register the fork as an enabled source, and optionally **open a Pull Request** back upstream.

Your folder name is always your GitHub login. To keep it that way, the **Identity** field in ⚙ **Configuration** → 🔑 **GitHub account** is **locked to your account login whenever a PAT is set** (re-derived from the token and stored on Save), and the publish path is re-derived from the token at publish time — so changing your display name can never create a second `users/<x>/` folder. Clear the PAT to edit the identity manually.

Read-only viewers can do more than you might expect: **fetch from GitHub and cloud, add comments, and locally curate their own copy** — **removing platforms and BBs** they don't want and **drag-reordering the platform columns** freely, just like an admin — all without signing in. Because read-only users never publish, these changes are purely local (they tailor your in-browser view and free storage); nothing is committed and no one else is affected. Re-fetch to bring removed items back.

---

4. The header and navigation

The header is grouped into compact menus to stay tidy:

- **About** — version, contact, logo, and header links. Also the quick-access hub for **Contributors**, **Help & FAQ**, and the **Activity log**.
- ⚙ **Configuration** — **GitHub account**, ☁ **Cloud storage**, **Manage lists**, and the **Status manager**. A small dot on this button shows GitHub connection state (solid = a token is set; hollow = none).
- **Distribution** — fetch / publish the shared dashboard, plus the duplicate resolvers.
- **Analyse ▾** — **Cross-platform analysis** and **AI Insight**.
- ⬇ **Download ▾** — scoped BB bundle, **Export BBs (xlsx)**, and **Backup (.json / .zip)**.
- ⤓ **Import ▾** — import a backup / `dashboard.json`, or run a GitHub fetch (admins).
- **Cleanup** — reset, clear files, clear cache.

Window navigation. Every window (modal) that opens another window shows ‹ **Back** and **Forward** › buttons next to Close, working just like a browser: Back steps to the previous window, Forward returns through them. Each appears only when there is somewhere to go.

Minimise. The ⌃ **Min** toggle collapses the header to just the search/filters and Cleanup, for a larger working area. Every dropdown menu repositions itself to stay fully on screen.

---

5. Building blocks and their tabs

Click a BB card to open its panel. The tabs are:

- **Details** — owner, platform, category, type, maturity (CDI / TRL), description with per-user **notes**, parameters, and the ☁ **Cloud data** panel.
- **Layout** — the sketch image and the GDS / OASIS / SVG layout (see §11).
- **S-matrix** — S-parameter datasets, organised in folders.
- **Tests** — measurement / characterisation files, in folders.
- **Fabrication** — fabrication data, in sub-tabs.
- **Scripts** — Python scripts you can edit and run in-browser.
- **Documents** — datasheets and manuals.
- **Variants** — sub-blocks (e.g. MMI v1 / v2 / v3).
- **AI Insight** — an auto-summary of the BB with citation-checked numbers.

Each tab header shows a count; counts include files nested inside subfolders.

---

6. Getting the data — Distribution

The dashboard ships as an empty shell; the live PDK comes from GitHub.

1. Header → ⍰ **Distribution**.
2. ⍰ **Fetch all enabled sources** pulls the published `dashboard.json` plus every contributor's `users/<id>/dashboard.json` slice. *Auto-pull user slices* is on by default; untick it for a clean admin-only snapshot.
3. Choose **Merge into my dashboard** (additive) or **Replace whole dashboard**. A local backup `.json` downloads first.

This works for public repos with **no GitHub account and no token**. Large `dashboard.json` files (over 1 MB) are fetched via the raw host automatically, and the importer falls back to a single repo-tree listing if GitHub's unauthenticated rate limit is hit.

A fresh install ships with two default **shared repositories** under *1. Shared repositories*: the **CORNERSTONE main** dashboard and a **User-admin contributions (auto-discover)** source that walks the `users/` tree. You can add, edit, or remove sources freely (one per team / fork); the defaults are seeded only once, so deletions stick.

---

7. Importing a platform from GitHub

On a platform column header, click ⤓ **Import platform**:

- Add one or more GitHub source URLs (admins / user-admins); read-only viewers run against admin-saved URLs.
- Pick which sections to fetch (Layout, S-matrix, Tests, Fab, Scripts, Documents).
- The importer walks `components/` and creates one BB per `*.gds`.

Two-source model (GitHub + cloud mirror). The import dialog has a ☁ **Cloud mirror** field with a ⍰ **Test** button and a *"Populate cloud links on each BB from this mirror"* checkbox (off by default). GitHub stays the **principal** source (catalogue + layout); the cloud mirror is an **assisting** source for heavy data — enabling it only *adds* cloud links, it never replaces what GitHub pulls. When populated, links mirror the repo's **actual** folder names (discovered from GitHub) nested per BB. The platform's ☁ **Cloud sync** button re-distributes those links without downloading.

---

8. Cloud storage — fetching large data

> **Why:** keep the catalogue in GitHub and the gigabytes of raw data in cloud storage. The dashboard fetches bytes on demand and caches them locally; only a small pointer travels in `dashboard.json`.

8.1 Where to set cloud links

- **Per file** — `☁ Link` on a dataset/test card (a direct URL or a SharePoint/OneDrive share link).
- **Per folder** — `☁ Cloud base URL` on an S-matrix / Tests / Fab folder; files resolve as `<base>/<filename>`.
- **Per section** — the `☁ Cloud links` button on each BB tab, next to `⤓ GitHub fetch`.
- **Whole BB** — the `☁ Cloud data` panel on the Details tab runs every section's links at once.

All of these are available to **read-only viewers** for fetching; only admins / user-admins can change the links.

8.2 Fetching

- A cloud-backed file shows **☁ Fetch**. Click it (or Download / Plot) and the bytes stream in and cache locally.
- **Default** fetch pulls each section's known files individually — including the common **single-file layout** where a section's data is one file named after the BB (e.g. `sparams/<bb>.csv`). It tries the base both as a file stem (`<base>.csv`) and as a folder (`<base>/<bb>.csv`).
- Tick **Fetch everything** (off by default; on the per-tab ☁ Cloud window and the Details panel) to instead **list the whole cloud folder, including subfolders**, recreate that structure under a clearly-marked **☁ Cloud** folder (with nested subfolders so it's obvious what came from cloud), and import **only files not already present**. Supported for **S3** (needs `s3:ListBucket`; recursive + paginated) and **SharePoint/OneDrive** (walks subfolders via Graph).
- **? Test** before committing a link: it reports CORS reachability, the file count and total size for a folder, and — for the single-file layout — confirms reachability of `<base>.csv`. It surfaces the real S3 error (e.g. `AccessDenied`) instead of a bare 403.

8.3 Provider notes

The fetch is a browser request, so the host **must allow cross-origin (CORS) reads**.

Amazon S3 — Bucket → Permissions → CORS:

```
[{ "AllowedOrigins": ["https://YOUR-DASHBOARD-ORIGIN"],
   "AllowedMethods": ["GET", "HEAD"],
   "AllowedHeaders": ["*"],
   "ExposeHeaders": ["Content-Length","Content-Range","ETag","Accept-Ranges"],
```

```
  "MaxAgeSeconds": 3000 }]
```

Make objects readable (public-read policy granting `s3:GetObject`, or pre-signed URLs). To auto-list a folder, also allow `s3:ListBucket`. A `403` after CORS is set means the object isn't readable (not a CORS problem).

Azure Blob / GCS / R2 / B2 / MinIO — enable CORS for your origin and use public or pre-signed URLs.

Dropbox / Google Drive — paste the normal share link; the dashboard converts it to a direct-download form (some consumer links don't send CORS headers).

SharePoint / OneDrive (private org data) — Header → ☁ **Cloud → Connect private SharePoint / OneDrive**. One-time Azure setup: register a **single-page application**, set the redirect URI to the dashboard's URL, add delegated Graph permissions `Files.Read.All` + `Sites.Read.All`, paste the **Client ID**, Save, then **Sign in**. Requires the dashboard served over **https**.

8.4 Design manuals from cloud

The platform's ? **Design manuals** window has a ☁ **Cloud fetch** button alongside ↻ **Sync from GitHub** — point it at a cloud folder to pull the manuals from there.

---

9. Cloud storage — uploading (admins)

The dashboard can also **push files into your own bucket**, not just read from it. In ⚙ **Configuration → ☁ Cloud storage**, admins/maintainers see ⬆ **Configure cloud upload**:

- **AWS S3** (and S3-compatible R2 / B2 / MinIO): region, bucket, optional custom endpoint, access key ID, secret, optional STS session token, and a default key prefix. Uploads are signed in-browser with **AWS Signature V4**.
- **Generic PUT endpoint** for an internal server or pre-signed prefix.
- Credentials are stored **only in this browser** (localStorage) — never written into the dashboard or published. Use ? **Test write** to confirm credentials and CORS allow `PUT`.

Then every S-matrix / Tests / Fabrication folder shows an admin-only ⬆ **Cloud upload** button that pushes that folder and its subfolders up, recreating the structure as key prefixes. Files without local bytes (un-fetched cloud stubs) are skipped. Your bucket's CORS must allow `PUT` and the IAM identity needs `s3:PutObject`.

---

10. Working with files in a BB

Inside the S-matrix, Tests, and Fabrication tabs, files live in folders (with optional subfolders). Each file card offers:

- **Plot** — plots the data (uses parsed data if present, otherwise parses the raw text on the fly).
- **▶ Run script** — run a Python script over this file.
- **View raw** — show the raw text.
- **⬇ Download** — download the original bytes back.
- **☁ Fetch / ☁ Link** — cloud controls (see §8).
- **Edit metadata / Rename / ×** — admin / owner only.

Folder-level actions include **+ Add files**, **+ Subfolder**, **Plot all raw**, **▶ Run script over folder**, **Run locally**, **↓ GitHub fetch**, **☁ Cloud links**, and **⬆ Cloud upload**.

Ownership is respected: you can always download another contributor's file, but only the owner or an admin can rename, edit, or delete it.

---

11. Layout / GDS viewer

The **Layout** tab toggles between the **Sketch** (an image fetched from the repo's `Sketch/` folder, uploaded, or picked from the shared library) and the **Layout (GDS/OAS/SVG)** view. GDS is parsed in-browser and rendered polygon-by-layer, with a clickable layer legend below the figure (click a chip to hide/show that layer). OASIS files are stored verbatim for round-trip publish/download.

Tools include **Measure**, **Set scale** (calibrate microns), **⬇ Download .gds / .svg**, and GitHub fetch / PR. The **⤢ Minimize tools** button hides all toolbar buttons except the Sketch / Layout toggles, giving the GDS view the full width.

---

12. Plots and figures

Plot windows (Spectrum and folder plots) are interactive Chart.js figures:

- **Drag** to pan, **wheel** to zoom, **shift+drag** to box-zoom, **Reset zoom** to restore.
- **Linear (raw)** / **dB** y-axis toggle.
- **+ Overlay dataset** to add another BB's traces; **Clear overlays** to remove them.
- **Minimize legend** — hides the legend box *and* the hover tooltip so the figure uses the whole canvas; click

again to restore.
- Export the figure as **PNG**, or the data as **CSV** / **XLSX**.

---

13. Scripts — editor and runner

Open a script (the source button on a script card) to get a full editor:

- **Line-numbered source** — a synced gutter so a Python traceback's "line N" maps straight to your code. (User code runs as its own unit, so line numbers match exactly — they are *not* shifted by the matplotlib bootstrap.)
- ▶ **Run** — executes the current source in-browser via Pyodide, auto-loading numpy / scipy / pandas / matplotlib as needed. **Running also saves the script**, so what runs is exactly what's stored.
- **Output console** — shows stdout, stderr (in red), the last expression, and any matplotlib plots. It clears at the start of each run so stale errors never linger.
- ⛶ **Maximize** — grows the console (shrinks the editor) for reading long output; ⧉ **Detach** pops the output into its own resizable browser window.

The same line-numbered editor and output appear in the **Run script** dialog used on individual data files and folders.

---

14. Comments and notes

Every S-matrix folder, test folder, fabrication sub-tab, and script has a 💬 **Comments** thread, and each BB's **Details** tab has **+ Add note**. Any contributor (including read-only) can add one — it's stamped with their name and date. Only the author or an admin can delete it.

Comments and Details notes **travel with publish/merge** and are **unioned across contributors** (de-duplicated by id), so when you fetch everyone's repos you see everyone's discussion — without any file contents riding along.

---

15. Uploads view

The 📤 **Uploads** window lists every uploaded folder across all BBs. **Layout files (GDS / OAS / SVG)** appear here under a 🧩 **Layout** section for each BB — traceable however they arrived (manual upload, GitHub fetch, or import).

Every file row has a 🗑 delete button (admins): deleting a layout entry clears that BB's layout slot; deleting any other file removes it, including from nested subfolders. You can also filter, download all as a zip, or save the bucket to a folder on your computer.

---

16. Finding and removing duplicates

From 🗑 **Distribution**:

- 🗑 **Resolve duplicates** folds BBs / platforms / tabs that share a name within the same parent (typically after merging dashboards), choosing a keeper and merging the rest.
- 🗑 **Resolve duplicate files** scans every BB for files that appear more than once (same name/size) across all tabs, subfolders, and uploaded files — tick the redundant copy and delete it while keeping the other.

A local backup `.json` downloads before any change.

---

17. Multi-select and bulk delete

- **BB lists** — each platform column header has a ☑ **Select all** (it appears once you tick a BB) for bulk 🗑 **Move** / 🗑 **Download**.
- **File lists** — every S-matrix / Tests / Fabrication folder has its own ☑ **Select all** and 🗑 **Delete selected** beside its **+ Add comment** button, scoped to that folder (and applied across subfolders for the file checkboxes). The flat Scripts tab has a single Select-all / Delete bar.
- **Contributors** (admin) — a Select-all column and 🗑 **Delete selected**; removals are buffered until you click 🗑 **Save**.
- **Sketch library** (admin / owners) — a checkbox per editable image with Select-all and bulk delete.

---

18. Analysis and AI Insight

Analyse ▾ → 🗑 Cross-platform analysis lets you select BBs across platforms, view distribution charts, run the Benchmark table, and export CSV / JSON / Markdown.

Analyse ▾ → 🗑 AI Insight is an LLM assistant: configure an API endpoint + key, then summarise an individual BB, a platform, or the whole dashboard. The pipeline extracts metrics deterministically in-browser and the LLM only

synthesises; every numeric claim is citation-checked.

---

19. Managing lists and statuses

⚙ **Configuration → ? Manage lists** edits the dashboard's reference lists: statuses, categories, BB types, EDA tools, CDI axes, platforms, layer map, benchmark metrics, and the **sketch library**. Read-only sessions can view; full Admin edits anything; user-admins manage their own items.

⚙ **Configuration → ? Status manager** opens the legacy-status repair tool: it lists every BB carrying an unrecognised status with Open / Diagnose / Remap / Force-purge actions, and lets you adopt an orphan status into your list. A dashed chip also appears in the stats row when legacy statuses exist.

---

20. Publishing and the repo-size model

- **Add BBs / files** in user-admin or admin mode (drag-drop, GitHub fetch, or cloud fetch).
- **Publish** via Distribution: a user-admin publishes a slice to `users/<login>/dashboard.json`; an admin publishes the canonical `dashboard.json`. The user-admin publish path is **always** `users/<your-github-login>/` (re-derived from your token), and admins are blocked from publishing into the `users/` subtree so a stray admin publish can't prune contributors' folders.
- **Direct commit vs. fork & PR.** Use ⚡ **Direct commit** if your PAT has push access to the source; otherwise ? **Fork & publish** forks the repo to your account, commits your slice there, and hands you a one-click **Open PR** to upstream. Either way your real files are uploaded under `users/<login>/<section>/<bb>/` and orphaned files (no longer in your local dashboard) are pruned, so the repo mirrors your local state. The folder itself is created automatically on first sign-in or first publish — there is no manual "seed" step.
- **The published `dashboard.json` is a catalogue — metadata only.** File bytes (S-matrix / tests / fab / uploads) and their parsed arrays are stripped on publish regardless of size; only filenames, sizes, cloud/GitHub pointers, comments, and notes travel. This keeps repos small even as data and discussion grow — adopters re-hydrate bytes on demand from cloud/GitHub. Small documents / YAML descriptors (under 512 KB) still round-trip. **Your local copy always keeps full bytes.**
- The **contributor list** (admin-managed) is the single source of who's who; edit it from About → ? **Contributors**.

Deletions propagate (tombstones). Merges are additive, so to stop a deleted item lingering forever in everyone's copy, the dashboard records a small *deletion record* when you remove a tab, platform, BB, or file. It travels in `dashboard.json` (and slices); on the next fetch+merge, adopters lose the deleted item too — **unless they own it**

or contributed to it (their own work is never wiped). So an admin can delete a tab and it disappears for everyone who hadn't built on it, while anyone who added their own BBs/files/comments to it keeps their contributions.

---

21. Backups, export, and cleanup

- **Backup user dashboards** (admin, in Distribution) — searches every `users/` folder across **all enabled sources live on GitHub** (never the local contributor list), lists each contributor that has a real `dashboard.json`, and lets you tick which ones to download. The selected slices come down as a single `.zip` laid out as `<owner>-<repo>/<user>/dashboard.json`, plus a `manifest.json` of every source scanned. It is entirely read-only — nothing on GitHub changes.
- **⬇ Download ▾ → Backup (.json / .zip)** — a full local backup. `.json` re-imports exactly; `.zip` is a browsable folder tree.
- **⬇ Download ▾ → Download (scope)** — bundle every matching BB (by platform / type / status) into a single zip.
- **⬇ Download ▾ → Export BBs (xlsx)** — an Excel workbook summarising every BB, platform, file inventory, tests, S-matrix, and contributors.
- **⤓ Import ▾** — import a backup or a fetched `dashboard.json`; admins can also run a GitHub fetch from here.
- **Cleanup** — reset to factory defaults, clear all uploaded files (keep the skeleton), or clear this browser's cache.

---

22. Tips and troubleshooting

- **"Nothing to fetch" in Distribution** — check the source path; for contributor slices ensure *auto-pull user slices* is ticked (it is by default).
- **403 from S3** — the object isn't readable; make it public or use a pre-signed URL. CORS only governs whether the browser can read the response.
- **Cloud Test says "0 files" but a file exists** — your data is probably a single file named after the BB; Test now also probes `<base>.csv`. If your data is in nested folders instead, tick **Fetch everything**.
- **Cloud upload fails** — the bucket CORS must allow `PUT` for this origin and the key/secret's IAM identity needs `s3:PutObject`. Use **Test write** to diagnose.
- **SharePoint sign-in fails** — confirm the dashboard is on https and the Azure redirect URI matches exactly.
- **I have two `users/` folders** — fixed in 1.1.0: the folder name is now locked to your GitHub login. If an old build left a stray `users/<old-name>/` folder, delete it on GitHub once (or sign in under that old identity

and use ⚠ **Wipe my repo folder**); future publishes only ever touch `users/<your-login>/`.

- **Sign-in only asks for a token now** — that's intended: contributor sign-in is your GitHub PAT, with no shared password. The owner admin password is the small section at the bottom of the Sign-in screen.
- **Script error points at a wrong line** — fixed in 1.1.0: tracebacks now match the editor's line numbers. If output looks stale, it clears on each run.
- **Pop-out / Detach window is blank** — allow pop-ups for the page.
- **Storage** — fetched cloud bytes cache in your browser (IndexedDB). Use the storage diagnostics in the header to monitor usage.

CORNERSTONE PDK Monitor — first designed and released by Emre Kaplan, University of Southampton. Maintained by the CORNERSTONE PDK team. Open to everyone — let's build the open integrated-photonics community together.